

Embedded Programming

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

2024-2025

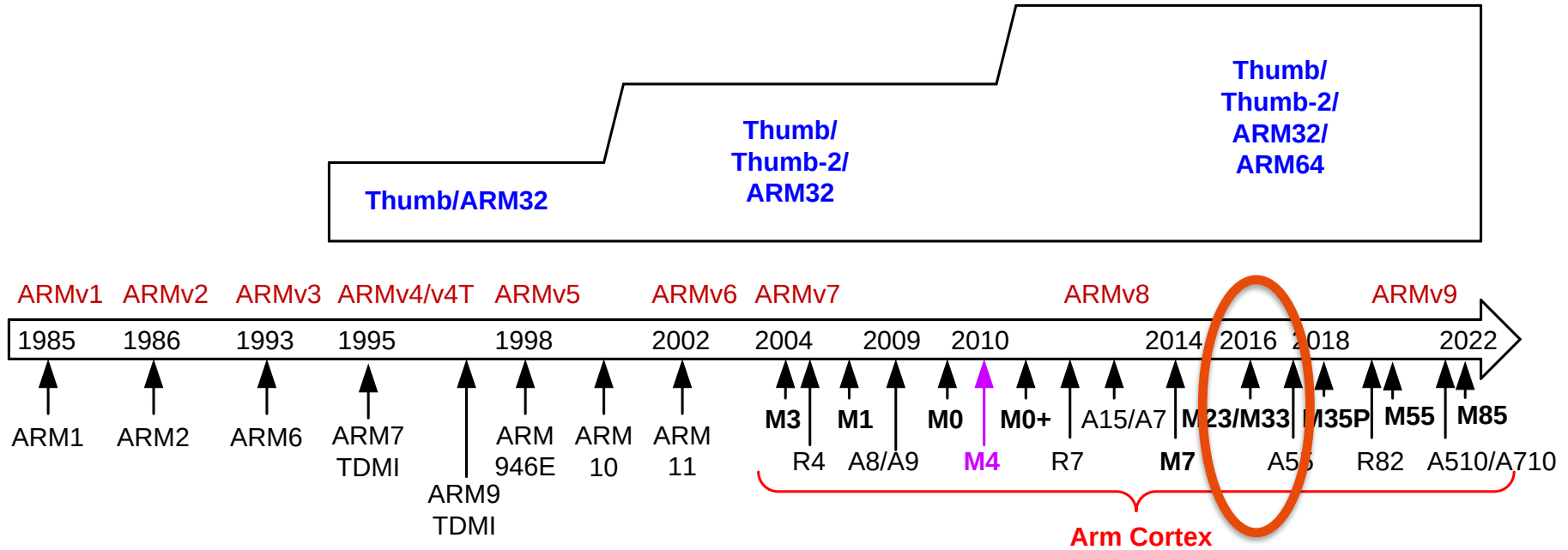
charis.kouzinopoulos@maastrichtuniversity.nl



Maastricht University

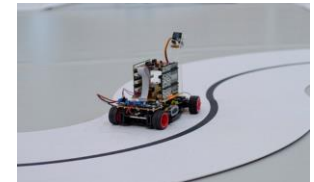
Introduction

History

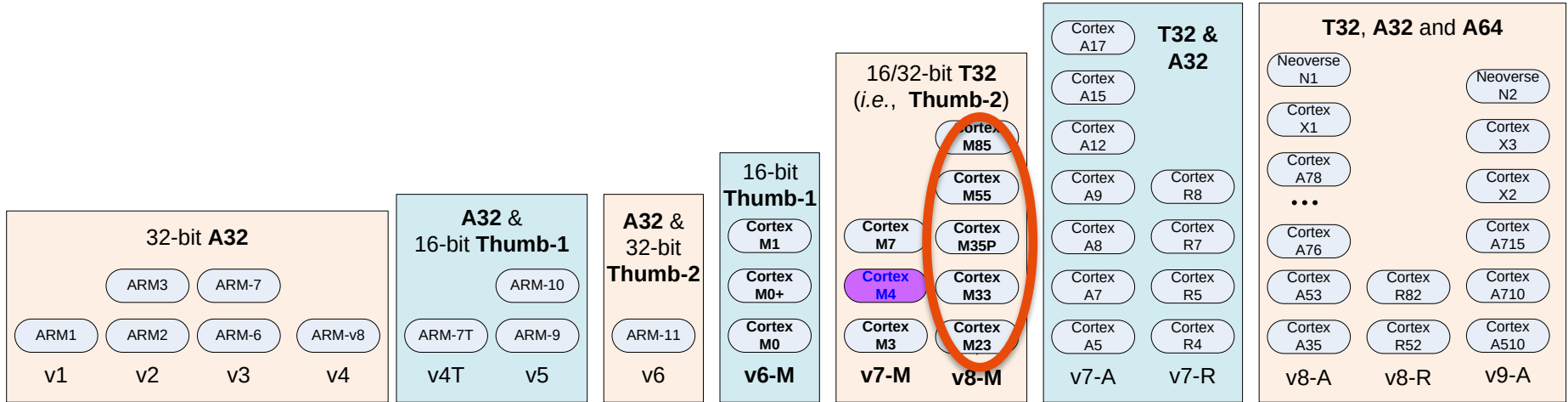


ARM Processors

- ARM Cortex-**A** family:
 - **A**pplications processors
 - Support OS and high-performance applications
 - Such as Smartphones, Smart TV
- ARM Cortex-**R** family:
 - **R**ead-time processors with high performance and high reliability
 - Support real-time processing and mission-critical control
- ARM Cortex-**M** family:
 - **M**icrocontroller
 - Cost-sensitive, support SoC

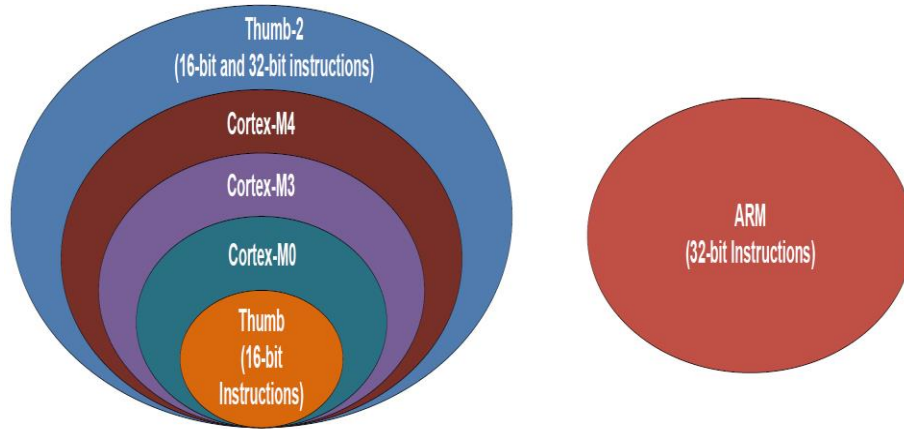


ARM Family



M: Microcontroller. **A:** Application. **R:** Real-time

Instruction Sets



- Instructions:
 - Encoded to binary machine code by assembler
 - Executed at runtime by hardware
- Early 32-bit ARM vs Thumb/Thumb-2
 - Early ARM has larger power consumption and larger program size
 - 16-bit Thumb, first used in ARM7TDMI processors in 1995
 - Thumb-2: a mix of 16-bit (high code density) and 32-bit (high performance) instructions
- ARM Cortex-M:
 - Subset of Thumb-2

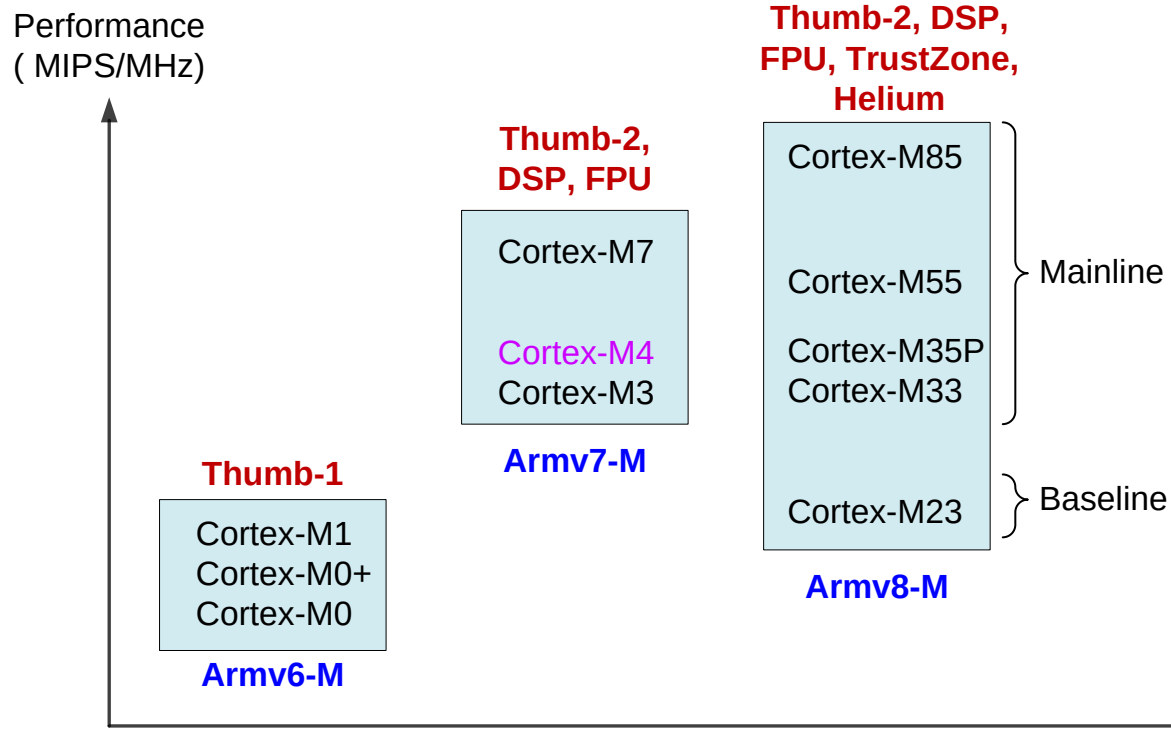
Instruction Sets

- ARM processors have 2 instruction sets, the traditional ARM set, where the instructions are all 32-bit long, and the more condensed Thumb (or T32) set, where most common instructions are 16-bit long (and some are 32-bit long)
- Which instruction set to run can be chosen by the developer, and only one set can be active (i.e. once the processor is switched to Thumb mode, all instructions will be decoded as using the Thumb instead of ARM)
- Although they are different instruction sets, they share similar functionality, and can be represented using the same assembly language. For example, the instruction

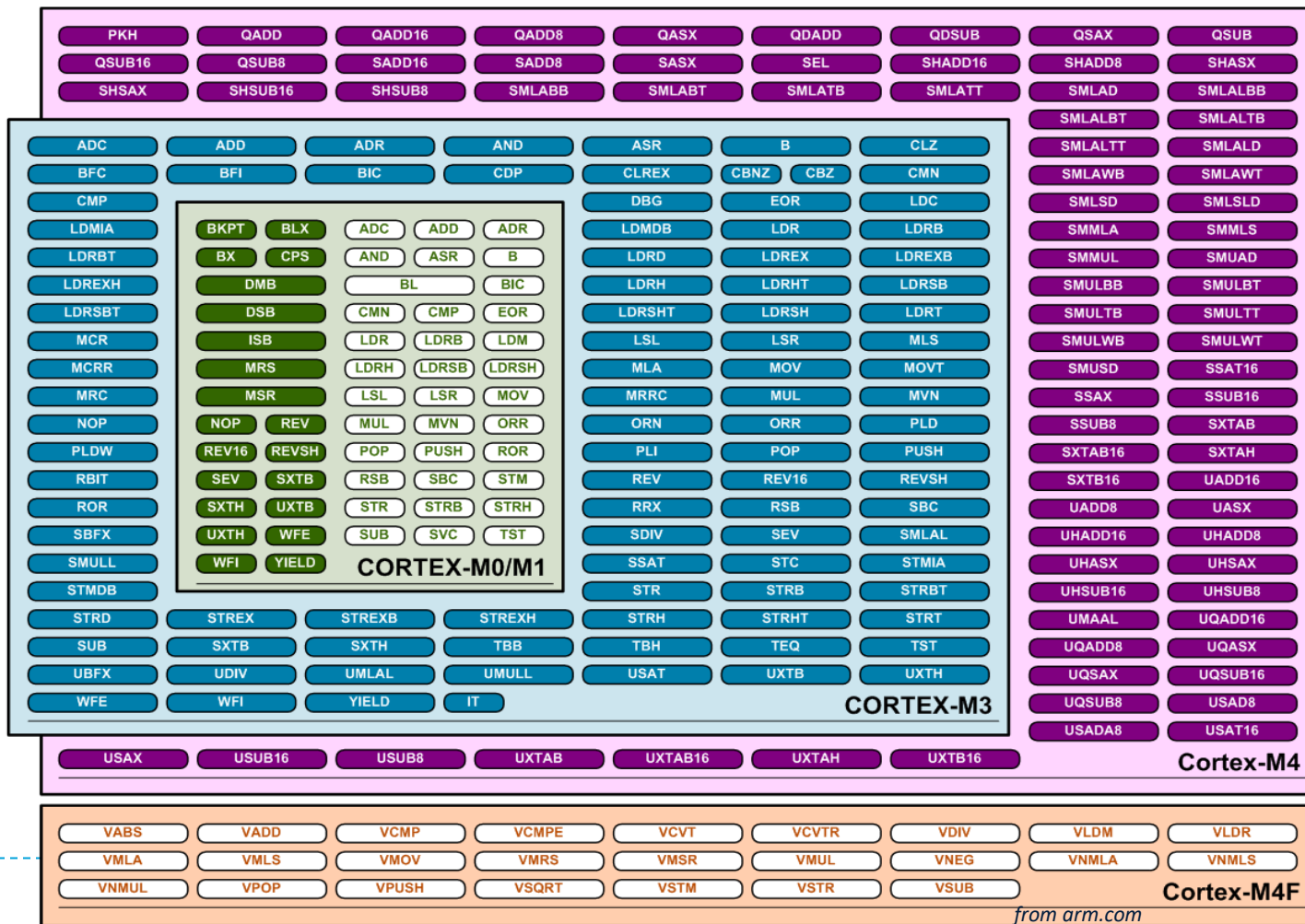
ADDS R0, R1, R2

can be compiled to ARM (E0910002 / 11100000 10010001 00000000 00000010) or Thumb (1888 / 00011000 10001000). Of course, they perform the same function (add r1 and r2 and store the result to r0), even if they have different encodings

ARM Processors



Instruction Sets

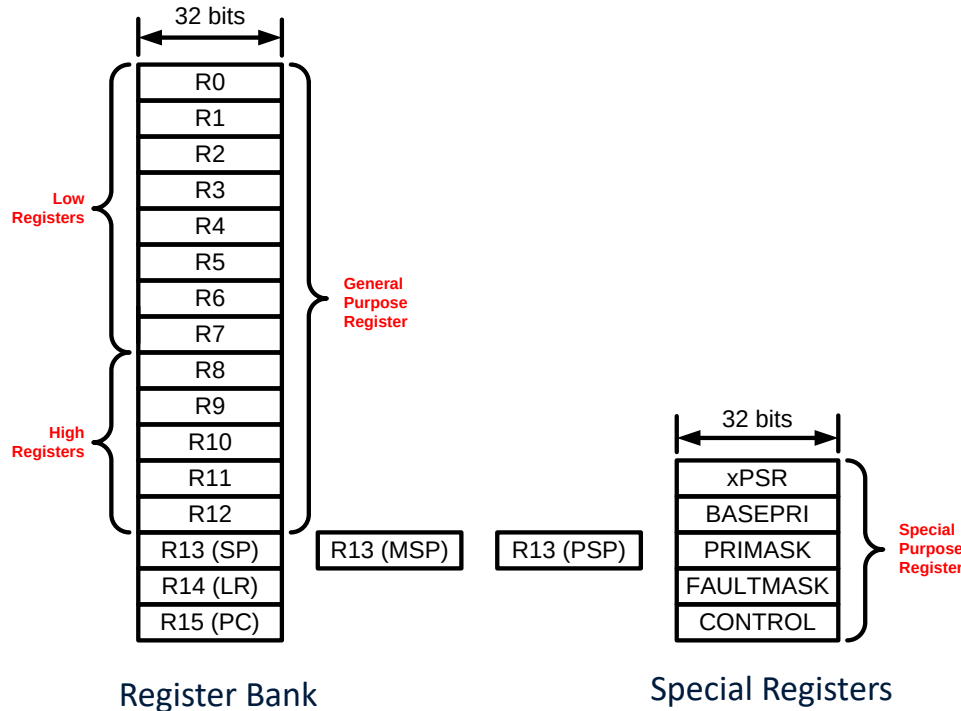


RISC vs SISC

- RISC – Reduced Instruction Set Computers
- CISC – Complex Instruction Set Computers
- Different architectures for doing the same operations
- Suppose you wanted to multiply two numbers in memory locations `mem0` and `mem1` and store the results back in `mem0`
 - Same result but the complexity of operations and the number of steps used in the two cases differ

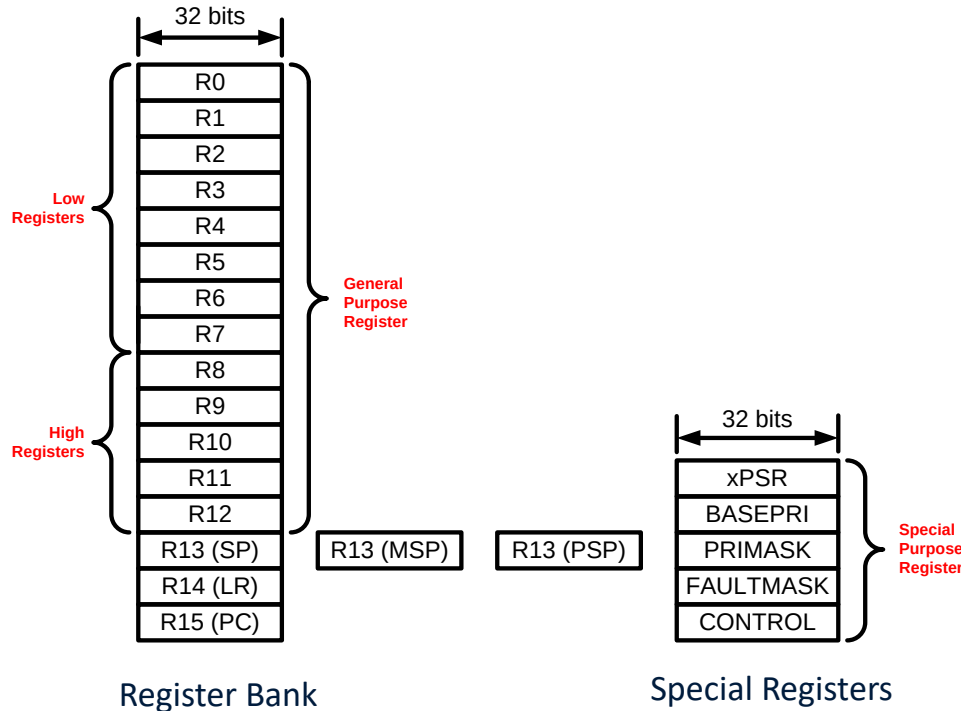
CISC Approach	RISC Approach
<code>mull mem0, mem1</code>	<code>ldr r0, mem0</code> <code>ldr r1, mem1</code> <code>mul r0, r0, r1</code> <code>str mem0, r0</code>

Processor Registers



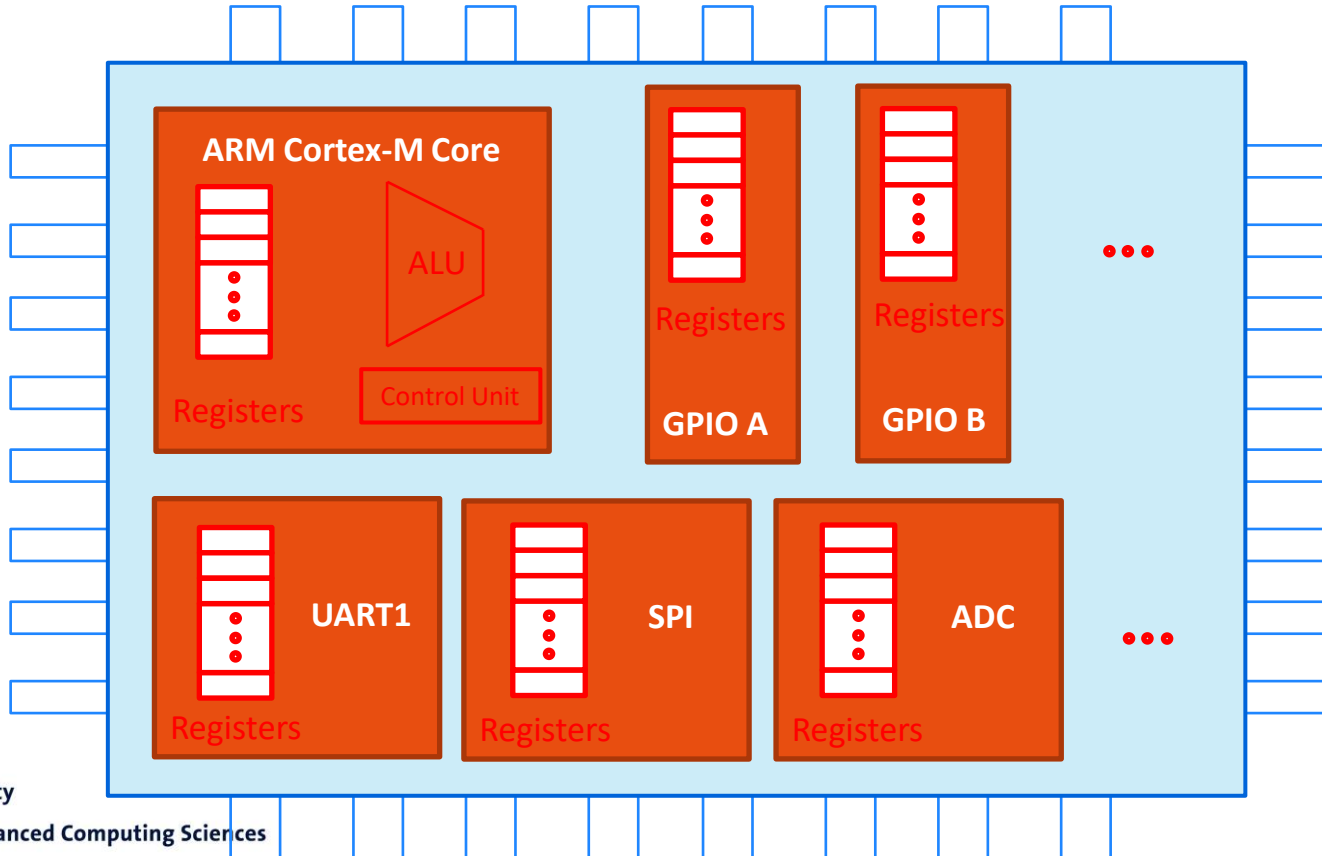
- ▶ Fastest way to read and write
- ▶ Registers are within the processor chip
- ▶ Each register has 32 bits
- ▶ ARM Cortex-M33 has
 - ▶ **Register Bank: R0 – R15**
 - ▶ **R0-R12**: 13 general-purpose registers
 - ▶ **R13**: Stack pointer (Shadow of MSP or PSP)
 - ▶ **R14**: Link register (LR)
 - ▶ **R15**: Program counter (PC)
 - ▶ **Special registers**
 - ▶ xPSR, BASEPRI, PRIMASK, etc

Processor Registers



- ▶ Low Registers (R0 – R7)
 - ▶ Can be accessed by any instruction
- ▶ High Registers (R8 – R12)
 - ▶ Can only be accessed by some instructions
- ▶ Stack Pointer (R13)
 - ▶ Cortex-M33 supports two stacks (four with the Security Extension)
 - ▶ Main SP (MSP) for privileged access (e.g. exception handler)
 - ▶ Process SP (PSP) for application access
- ▶ Program Counter (R15)
 - ▶ Memory address of the current instruction

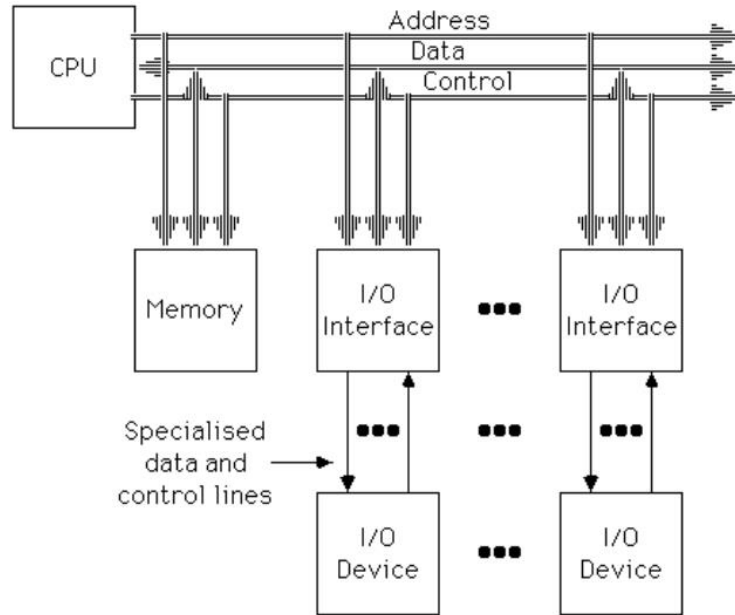
Processor Registers vs Peripheral Registers



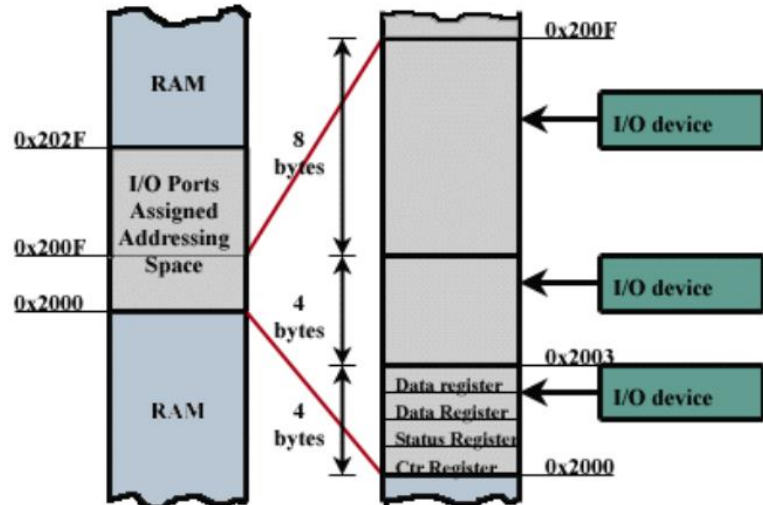
Processor Registers vs Peripheral Registers

- Processor can directly access processor registers
 - `ADD r3,r1,r0 ; r3 = r1 + r0`
- Processor access peripheral registers via **memory mapped I/O**
 - Each peripheral register is assigned a fixed memory address at the chip design stage
 - Processor treats peripheral registers the same as data memory
 - Processor uses load/store instructions to read from/write to memory

Memory mapping



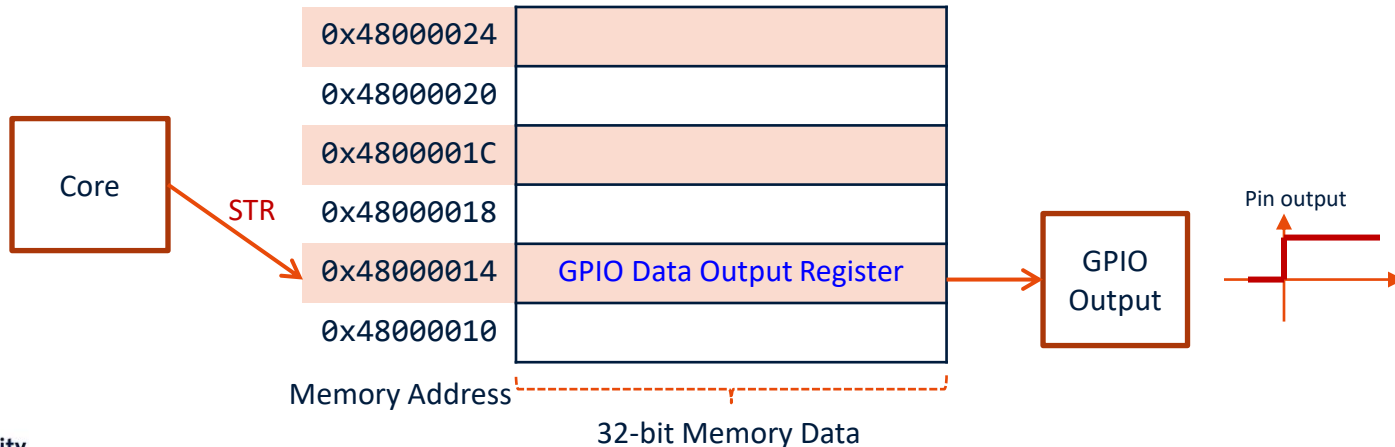
Physical Layout



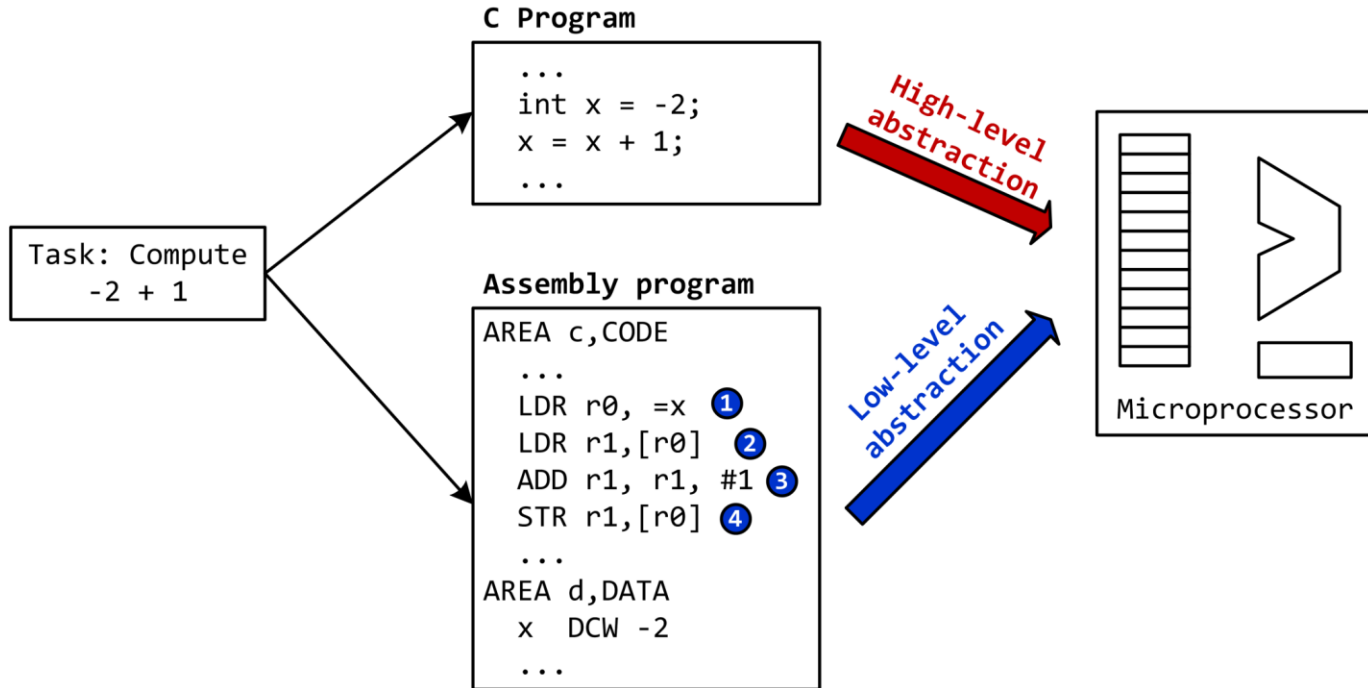
Programmer's View

Memory mapping

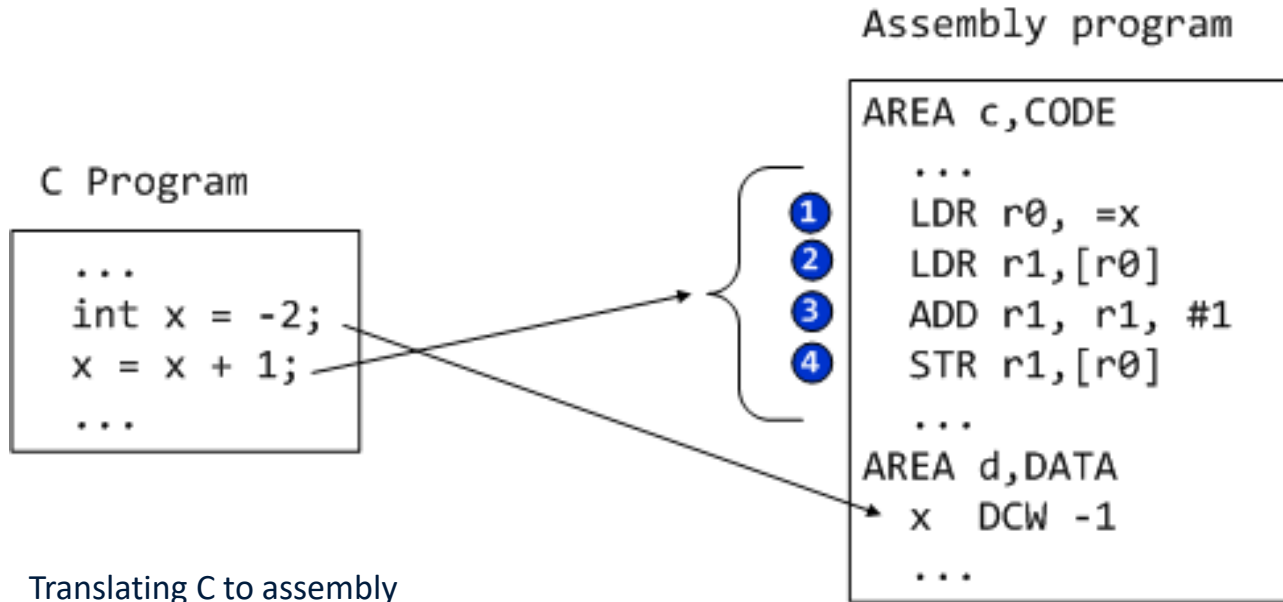
- **Port-mapped I/O**
 - Use special CPU instructions: `Special_instruction Reg, Port`
- **Memory-mapped I/O**
 - A simpler and more convenient way to interface I/O devices
 - Each device registers is assigned to a memory address in the address space of the microprocessor
 - Use native CPU load/store instructions: `LDR/STR Reg, [Reg, #imm]`



C vs Assembly



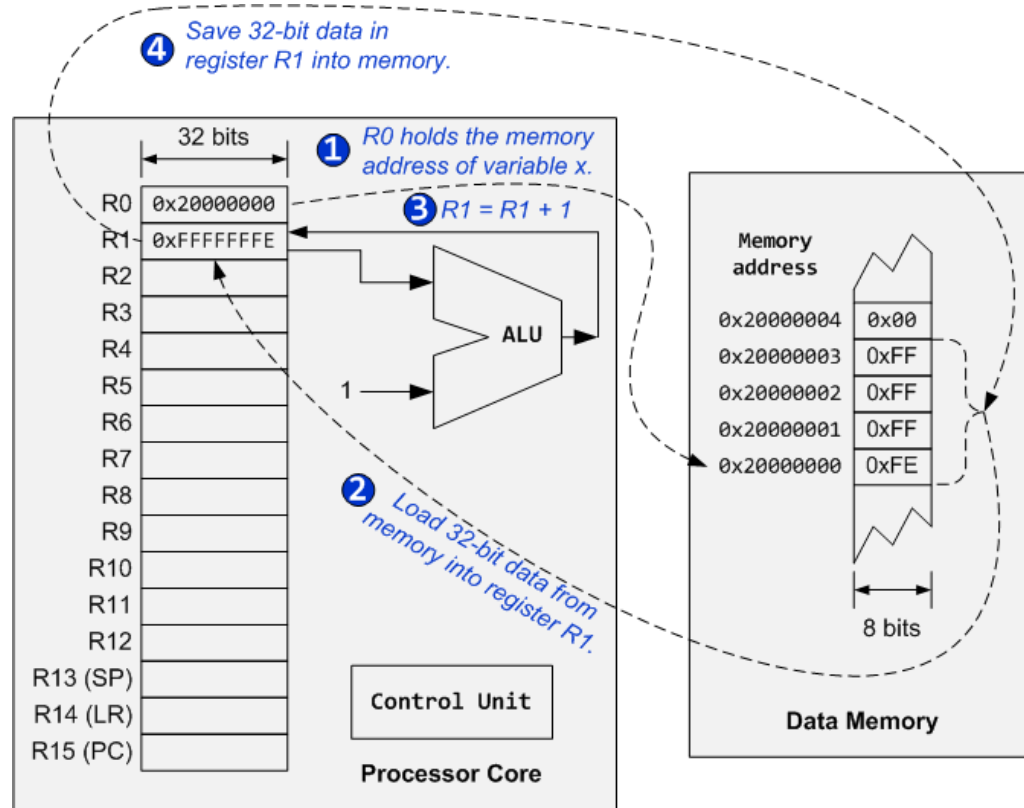
Load-Modify-Store



Translating C to assembly

- **Load** values from memory into registers
- **Modify** value by applying arithmetic operations
- **Store** result from register to memory

Load-Modify-Store



ARM assembly

Arithmetic and logical instructions



Assembly Instructions

- Arithmetic and logic
 - Add, Subtract, Multiply, Divide, Shift, Rotate
- Data movement
 - Load, Store, Move

Instruction Format: Labels

```
label    mnemonic operand1, operand2, operand3 ; comments
```

Instruction Format: Labels

```
label    mnemonic operand1, operand2, operand3 ; comments
```

- ▶ Place marker, marking the memory address of the current instruction
- ▶ Used by branch instructions to implement **if-then** or **goto**
- ▶ Must be unique

Instruction Format: Mnemonic

```
label1  mnemonic operand1, operand2, operand3 ; comments
```

- ▶ The name of the instruction
- ▶ Operation to be performed by processor core

Instruction Format: Operands

label mnemonic operand1, operand2, operand3 ; comments

- ▶ Operands
 - ▶ Registers
 - ▶ Constants (called *immediate values*)
- ▶ Number of operands varies
 - ▶ No operands: DSB ; Data Synchronization Barrier
 - ▶ One operand: BX LR ; branch and exchange instruction set, treat the next instruction as ARM or thumb
 - ▶ Two operands: CMP R1, R2
 - ▶ Three operands: ADD R1, R2, R3
 - ▶ Four operands: MLA R1, R2, R3, R4 ; multiply the values from R2 and R3, adds the value from R4, and places the least significant 32 bits of the result in R1
- ▶ Normally
 - ▶ operand1 is the destination register, and operand2 and operand3 are source operands.
 - ▶ operand2 is usually a register, and the first source operand
 - ▶ operand3 may be a register, an immediate number, a register shifted to a constant number of bits, or a register plus an offset (used for memory access)

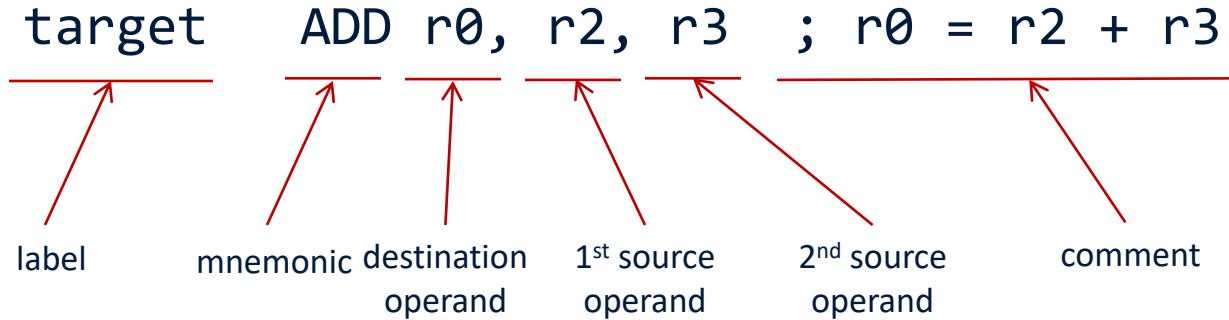
Instruction Format: Comments

```
label    mnemonic operand1, operand2, operand3 ; comments
```

- ▶ Everything after the semicolon (;) is a comment
- ▶ Explain programmers' intentions or assumptions

ARM Instruction Format

```
label    mnemonic operand1, operand2, operand3    ; comments
```



ARM Instruction Format

```
label    mnemonic operand1, operand2, operand3    ; comments
```

```
target    ADD r0, r2, r3    ; r0 = r2 + r3
```



Bad comment!

A better example:

```
; Increment angle r2 by step size r3
```

ARM Instruction Format

```
label    mnemonic operand1, operand2, operand3    ; comments
```

Examples: Variants of the ADD instruction

```
ADD r1, r2, r3    ; r1 = r2 + r3
```

```
ADD r1, r3        ; r1 = r1 + r3
```

```
ADD r1, r2, #4    ; r1 = r2 + 4
```

```
ADD r1, #15       ; r1 = r1 + 15
```

Adding Two Integers

```
int x = 1;  
int y = 2;  
int z;
```

If values are in registers

- ▶ Value of x in r0
- ▶ Value of y in r1
- ▶ Value of z in r2

C Statement

```
z = x + y;
```

Assembly Statement

?

Adding Two Integers

```
int x = 1;  
int y = 2;  
int z;
```

If values are in registers

- ▶ Value of x in r0
- ▶ Value of y in r1
- ▶ Value of z in r2

C Statement

```
z = x + y;
```

Assembly Statement

```
ADD r2, r1, r0
```

Destination

Source Operand 2

Source Operand 1

Adding Two Integers

```
uint x = 1;  
uint y = 2;  
uint z;
```

If values are in registers

- ▶ Value of `x` in `r0`
- ▶ Value of `y` in `r1`
- ▶ Value of `z` in `r2`

C Statement

```
z = x + y;
```

Assembly Statement

?

Adding Two Integers

```
uint x = 1;  
uint y = 2;  
uint z;
```

If values are in registers

- ▶ Value of x in r0
- ▶ Value of y in r1
- ▶ Value of z in r2

C Statement

```
z = x + y;
```

Assembly Statement

```
ADD r2, r1, r0
```

ADD works for both signed and unsigned add operations.

Adding Two Integers

```
int x = 1;  
int y = 2;  
int z;
```

C Statement

```
z = x + y;
```

If addresses are in registers

- ▶ Address of x in r0
- ▶ Address of y in r1
- ▶ Address of z in r2

Assembly Statements

?

Adding Two Integers

```
int x = 1;  
int y = 2;  
int z;
```

C Statement

```
z = x + y;
```

If addresses are in registers

- ▶ Address of x in r0
- ▶ Address of y in r1
- ▶ Address of z in r2

Assembly Statements

```
LDR r3, [r0] ; Read x  
LDR r4, [r1] ; Read y  
ADD r5, r3, r4  
STR r5, [r2] ; Write z
```

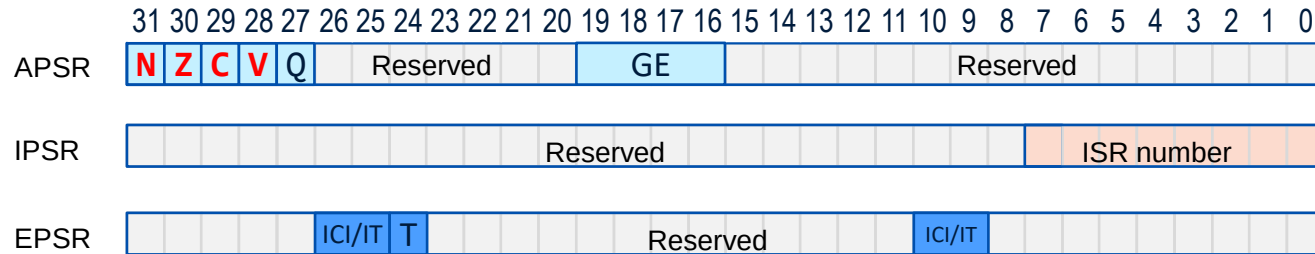
Load, modify, and store

Example Arithmetic Instructions

- **ADD** `r0, r1, r2` ; $r0 = r1 + r2$
- **ADC** `r0, r1, r2` ; Add with carry, $r0 = r1 + r2 + \text{carry}$
- **SUB** `r0, r1, r2` ; $r0 = r1 - r2$
- **SBC** `r0, r1, r2` ; Subtract with borrow, $r0 = r1 - r2 - (1 - \text{carry})$
- **MUL** `r0, r1, r2` ; $r0 = r1 * r2$, product limited to 32 bits
- **UDIV** `r0, r1, r2` ; Unsigned divide, $r0 = r1 / r2$
- **SDIV** `r0, r1, r2` ; Signed divide, $r0 = r1 / r2$
- **SMULL** `r0, r1, r2, r3` ; Signed multiply (64-bit product), $r1:r0 = r2 * r3$
- **UMULL** `r0, r1, r2, r3` ; Unsigned multiply (64-bit product), $r1:r0 = r2 * r3$

Program Status Register (PSR)

- Application PSR (**APSR**), Interrupt PSR (**IPSR**), Execution PSR (**EPSR**)



Updating NZCV flags in PSR

Flags not changed		Flags updated
ADD	→	ADD S
SUB	→	SUB S
MUL	→	MUL S
UDIV	→	UDIV S
AND	→	AND S
ORR	→	ORR S
LSL	→	LSL S
MOV	→	MOV S

*Most instructions update NZCV flags
only if S suffix is present*

`CMP r1, r2` vs `SUBS r0, r1, r2`

Some instructions update NZCV flags even if no S is specified.

- **CMP**: Compare, like SUBS but without destination register
- **CMN**: Compare Negative, like ADDS but without destination register
- **TST**: Test, like ANDS but without destination register
- **TEQ**: Test equivalence, like EORS but without destination register

ADD vs ADDS

ADD r0, r1, r2 ; r0 = r1 + r2, NZCV flags unchanged

ADDS r0, r1, r2 ; r0 = r1 + r2, NZCV flags updated

- ADD does not update flags
- ADDS updates flags
 - xPSR.**N** = bit 31 of result
 - xPSR.**Z** = IsZero(result)
 - xPSR.**C** = carry, assuming r1 and r2 representing unsigned integers
 - xPSR.**V** = overflow, assuming r1 and r2 representing signed integers

Suffix S: Update Flags

```
LDR r0, =0xFFFFFFFF
LDR r1, =0x00000001
ADDS r0, r0, r1
```

0xFFFFFFFF	r0
+ 0x00000001	r1
0x00000000	sum

N (Negative) = 0
Z (Zero) = 1
C (Carry) = 1
V (oVerflow) = 0

The screenshot shows a disassembler interface with two main panes. The left pane, titled 'Registers', displays a list of registers (R0-R15, xPSR) and their current values. R0 is 0xFFFFFFFF, R1 is 0x00000001, and R15 (PC) is 0x08000136. The right pane, titled 'Disassembly', shows assembly instructions. The instruction at address 0x08000136 is highlighted in yellow: `0x08000136 E7FE B stop 0x08000136`. Below this, the assembly code for the `main` function is visible, including the `ADD r3, r0, r1` instruction at address 0x0800013A.

Register	Value
R0	0xFFFFFFFF
R1	0x00000001
R2	0x00000000
R3	0x00000000
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x20000600
R14 (LR)	0xFFFFFFFF
R15 (PC)	0x08000136
xPSR	0x61000000
N	0
Z	1
C	1
V	0
Q	0
T	1
IT	Disabled
ISR	0

```
29: ADDS r3, r0, r1
30:
0x08000134 1843 ADDS r3, r0, r1
31: stop B stop
0x08000136 E7FE B stop 0x08000136
0x08000138 0000 MOVS r0, r0
0x0800013A 0000 MOVS r0, r0
0x0800013C 0000 MOVS r0, r0

main.s stm321xx_constants.s startup_stm321xx_md.s
1 ;***** (C) Yifeng ZHU *****
2 ; @file main.s
3 ; @author Yifeng Zhu
4 ;*****
5
6 INCLUDE stm321xx_constants.s
7
8 AREA main, CODE, READONLY
9 EXPORT __main
10 ENTRY
11
12 __main PROC
13
14 LDR r0, =0xFFFFFFFF
15 LDR r1, =0x00000001
16 ADDS r3, r0, r1
17
18 stop B stop
19
20 ENDP
21 ALIGN
22 END
```


Suffix S:

Update Flags

```
LDR  r0, =0xFFFFFFFF
LDR  r1, =0x00000001
ADDS r0, r0, r1
```

0xFFFFFFFF	r0
+ 0x00000001	r1
0x00000000	sum

N (Negative) = 0

Z (Zero) = 1

C (Carry) = 1

V (oVerflow) = 0

Result not negative

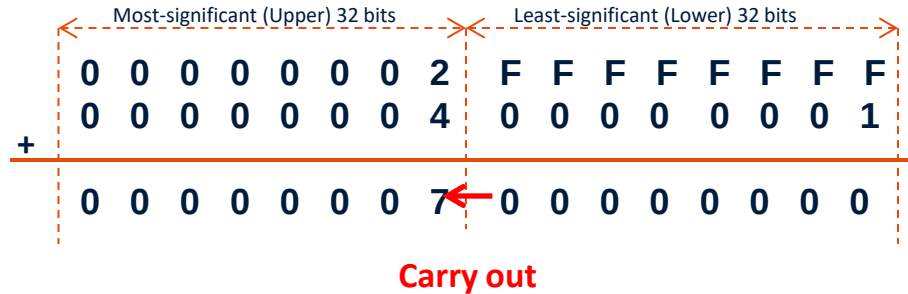
Result zero

There is a carry out of the most significant bit (MSB). Since adding 1 to 0xFFFFFFFF produces 0x00000000 with a carry out from bit 31 (the highest bit), the C is set to 1

The V flag is only set when a signed addition results in an incorrect sign change, which happens when:

- Adding two positive numbers results in a negative number
- Adding two negative numbers results in a positive number

Example: 64-bit Addition



- A register can only store 32 bits
- A 64-bit integer needs two registers
- Split 64-bit addition into two 32-bit additions

Example: 64-bit Addition

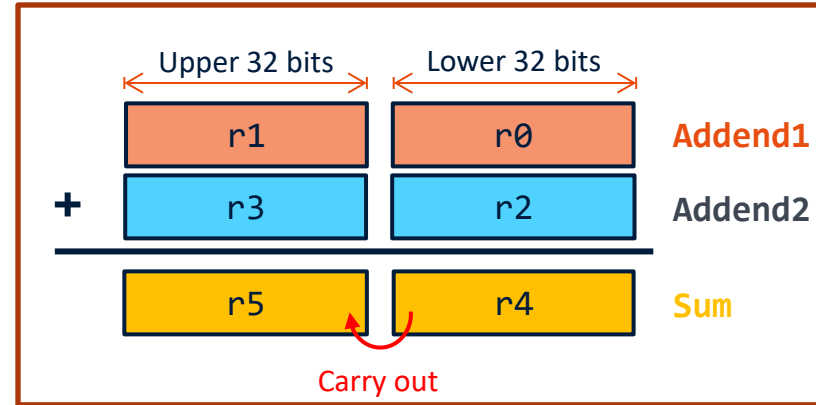
start

```
; C = A + B
; Two 64-bit integers A (r1,r0) and B (r3,r2).
; Result C (r5, r4)
; A = 00000002FFFFFFFF
; B = 0000000400000001
LDR r0, =0xFFFFFFFF ; A's lower 32 bits
LDR r1, =0x00000002 ; A's upper 32 bits
LDR r2, =0x00000001 ; B's lower 32 bits
LDR r3, =0x00000004 ; B's upper 32 bits
```

; Add A to B

```
ADDS r4, r2, r0 ; C[31..0] = A[31..0] + B[31..0], update Carry
```

```
ADC r5, r3, r1 ; C[64..32] = A[64..32] + B[64..32] + Carry
```



ARM assembly

Bitwise logic

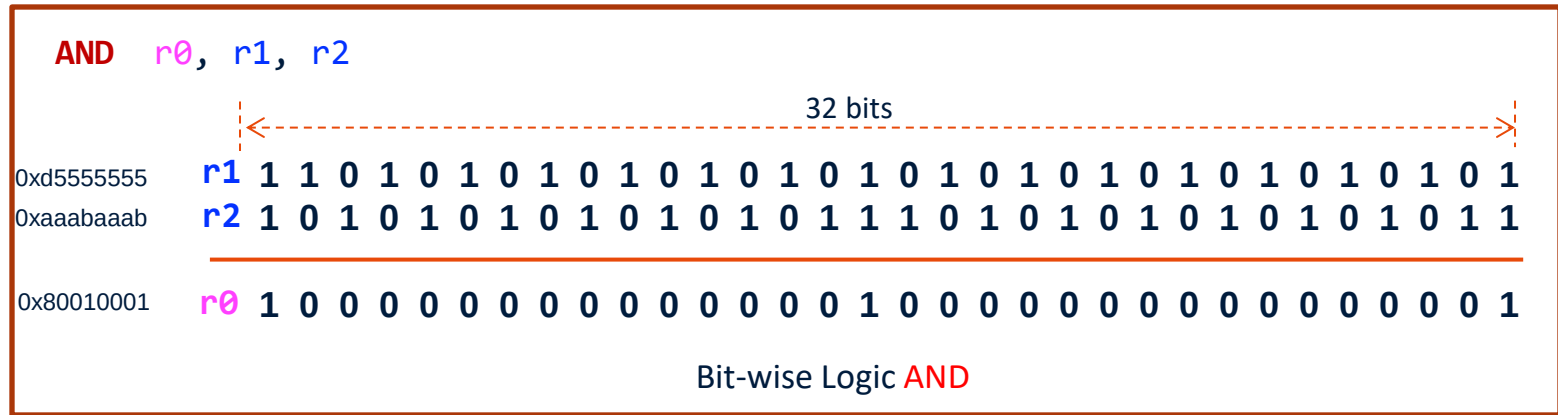


Bitwise Logic

AND {Rd,} Rn, Op2	Bitwise logic AND $Rd \leftarrow Rn \& \text{operand2}$
ORR {Rd,} Rn, Op2	Bitwise logic OR $Rd \leftarrow Rn \mid \text{operand2}$
EOR {Rd,} Rn, Op2	Bitwise logic exclusive OR $Rd \leftarrow Rn \wedge \text{operand2}$
ORN {Rd,} Rn, Op2	Bitwise logic NOT OR $Rd \leftarrow Rn \mid (\text{NOT operand2})$
BIC {Rd,} Rn, Op2	Bit clear $Rd \leftarrow Rn \& \text{NOT operand2}$
BFC Rd, #lsb, #width	Bit field clear $Rd[(\text{width}+\text{lsb}-1):\text{lsb}] \leftarrow 0$
BFI Rd, Rn, #lsb, #width	Bit field insert $Rd[(\text{width}+\text{lsb}-1):\text{lsb}] \leftarrow Rn[(\text{width}-1):0]$
MVN Rd, Op2	Move NOT, logically negate all bits $Rd \leftarrow 0xFFFFFFFF \text{ EOR } Op2$

Example Logical Instructions

- **AND** `r0, r1, r2` ; Bitwise AND, $r0 = r1 \text{ AND } r2$
- **ORR** `r0, r1, r2` ; Bitwise OR, $r0 = r1 \text{ OR } r2$
- **EOR** `r0, r1, r2` ; Bitwise Exclusive OR, $r0 = r1 \text{ EOR } r2$
- **ORN** `r0, r1, r2` ; Bitwise OR NOT, $r0 = r1 \text{ ORN } r2$
- **BIC** `r0, r1, r2` ; Bit clear, $r0 = r1 \& \sim r2$



Example: AND r2, r0, r1

	32 bits																															
r0	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
r1	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1
r2	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Bit-wise Logic AND

Example: ORR r2, r0, r1

32 bits

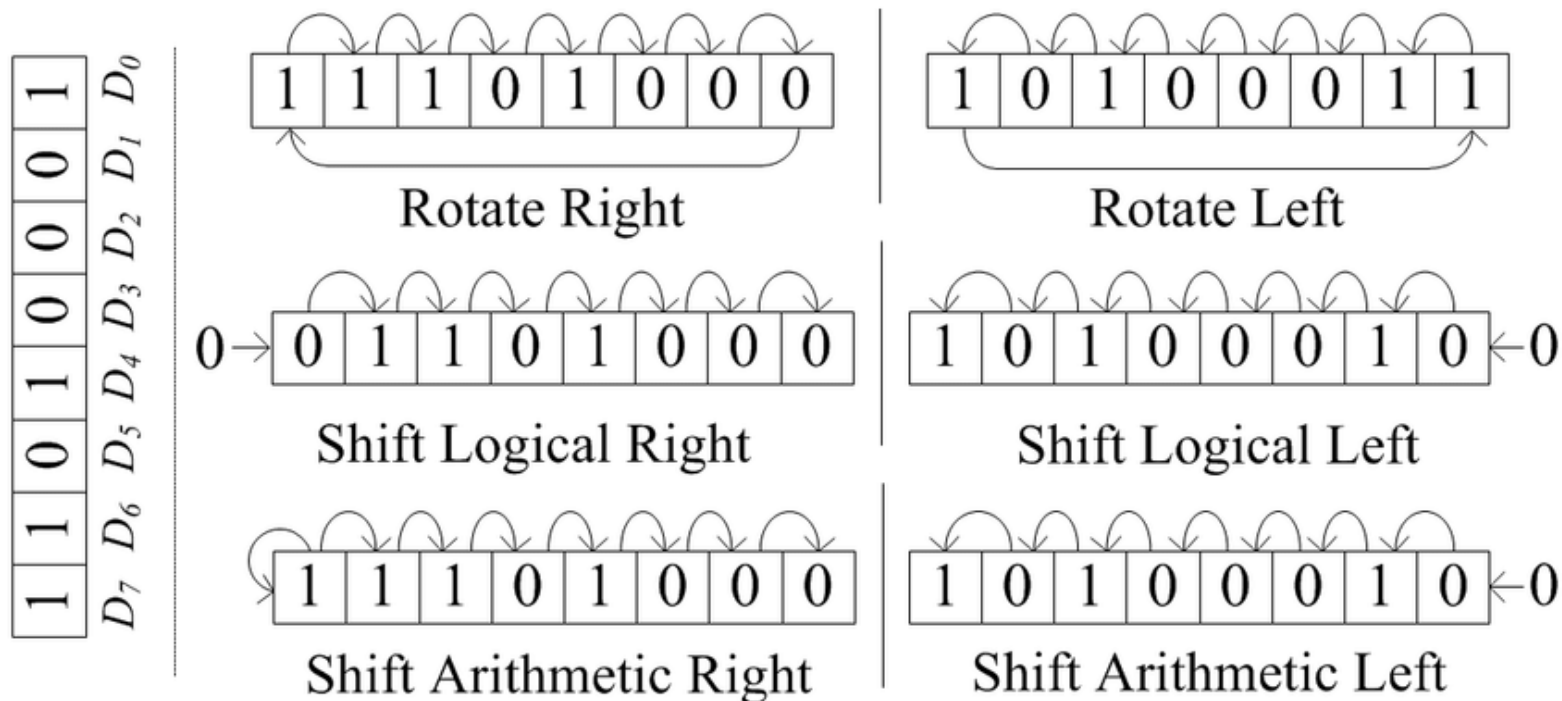
r0	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
r1	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1
r2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit-wise Logic OR

Results for ORR, EOR, ORN and BIC?

- **AND** `r0, r1, r2` ; Bitwise AND, $r0 = r1 \text{ AND } r2$
- **ORR** `r0, r1, r2` ; Bitwise OR, $r0 = r1 \text{ OR } r2$
- **EOR** `r0, r1, r2` ; Bitwise Exclusive OR, $r0 = r1 \text{ EOR } r2$
- **ORN** `r0, r1, r2` ; Bitwise OR NOT, $r0 = r1 \text{ ORN } r2$
- **BIC** `r0, r1, r2` ; Bit clear, $r0 = r1 \text{ \& } \sim r2$

Shift and Rotate



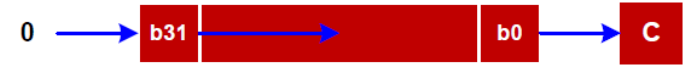
Example Shift and Rotate Instructions

- **LSL** `r0, r1, r2 ; Logical shift left,`
`r0 = r1 << r2`
- **LSR** `r0, r1, r2 ; Logical shift right,`
`r0 = r1 >> r2`
- **ASR** `r0, r1, r2 ; Arithmetic shift right,`
`r0 = r1 >> r2`
- **ROR** `r0, r1, r2 ; Rotate right,`
`r0 = r1 rotate by r2 bits`

Logical Shift Left (LSL)



Logical Shift Right (LSR)



Arithmetic Shift Right (ASR)



Rotate Right (ROR)



ARM assembly

Load / store / move

Load Instructions

- **LDR rt, [rs]**
 - **Read from memory**
 - Mnemonic: LoaD to Register (**LDR**)
 - rs specifies the memory address
 - rt holds the 32-bit value fetched from memory
- For Example:

```
; Assume r0 = 0x08200004
; Load a word:
LDR r1, [r0]          ; r1 = Memory.word[0x08200004]
```

Store Instructions

- **STR rt, [rs]**
 - **Write into memory**
 - Mnemonic: STore from Register (**STR**)
 - rs specifies memory address
 - Save the content of rt into memory
- For Example:

```
; Assume r0 = 0x08200004
; Store a word
STR r1, [r0]      ; Memory.word[0x08200004] = r1
```

Example Data Transfer Instructions

- **MOV** r0, r1 ; Move, r0 = r1
- **MVN** r0, r1 ; Move NOT, r0 = bitwise NOT r1

MVN r0, r1

r1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
r	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0
0																													

Bit-wise Logic **NOT**

Takeaways

- ARM processors (-A, -R, -M) and the ARM ISA (Thumb-1 and Thumb-2)
- Memory mapping and Load-Modify-Store
- Instruction format
- Arithmetic Instructions (ADD, ADC, SUB, SBC)
- Logical Instructions (AND, ORR, EOR, ORN, BIC)
- Shift and rotate (LSL, LSR, ASR, ROR, RRX)
- Program Status Register and the NZCV flags
- Data movement
- Bitwise operations